

2VG3 Tanks Hotseat Write-Up

Author: Kyle Drury

March 19, 2026

Preamble: Resources, Assets, and Implementation Notes

This project was built in Python using Panda3D 1.10.16 and its Bullet Physics integration. The game is presented as a 2D side-view artillery game, but internally it runs as a constrained 3D simulation. An orthographic camera is used to create the flat side-view look, and the rigid bodies are restricted to the X/Z plane so the gameplay behaves like a 2D game while still benefiting from Bullet's rigid body system, contact handling, and terrain collision.

The game is designed as a *hotseat* game. In this context, that means both players share the same computer and play locally on the same keyboard instead of using networking, split machines, or AI turns. The goal was to make the demo immediately playable in a classroom setting: two people can sit at one computer, each use their assigned half of the keyboard, and start a match without any additional setup.

The visual assets used by the game are intentionally simple and mostly come from local primitive-style Panda3D models stored in the project under `assets/models/panda3d`. In particular, the tank bodies are assembled from reused cube geometry, and the projectile visuals use a sphere model. No external premade tank model was imported. Instead, each tank was built directly in code as a stylized composite object: a lower hull for the tread section, an upper body block, a turret base, and a long barrel. On the physics side, each tank uses a compound rigid body made from two box shapes, which keeps the collision model simple and stable enough for gameplay tuning. This made it easier to control contact behavior, jumping, knockback, and tank-versus-tank collisions than if a more detailed mesh collider had been used.

The terrain is also generated entirely in code rather than loaded from an art asset. A one-dimensional height profile is produced from blended sine waves, smoothed, softened near the edges, and flattened at the spawn points so both tanks start on playable ground. That profile is then duplicated across the terrain depth to create a heightfield. The same height data drives both the visible terrain and the Bullet collision shape, which is important because shell impacts can deform the ground during play by carving craters into the heightfield. As a result, the environment is not just decoration; it is a mutable gameplay surface.

At a high level, the project is therefore best described as a small “2.5D” artillery game: it looks like a 2D duel, but it relies on a constrained 3D engine underneath. That design choice

made it possible to use Panda3D and Bullet effectively without needing a fully custom 2D physics system.



Figure 1: Gameplay overview showing the orthographic side-view camera, procedurally generated terrain, HUD, projectiles, and destructible crater system.

Exercise 2

How to play and goal: This demo is a two-player hotseat tank game played by two people on the same keyboard. Player 1 uses **A/D** to move, **W/S** to aim, **Q** for rapid fire, **E** for heavy fire, and **Left Shift** to jump. Player 2 uses **J/L** to move, **I/K** to aim, **U** for rapid fire, **O** for heavy fire, and **Right Shift** to jump. The goal is to reduce the opposing tank to 0 HP or force it to fall off the edge of the terrain. Press **R** to restart the round.

Exercise 3

Key physics from Bullet: The game uses Panda3D's Bullet physics engine for rigid body simulation, projectile motion, terrain collision, and contact response. Gravity is increased on the Z axis to -28 so shells arc more quickly and the match feels more active. Tanks and shells are constrained to the X/Z plane with Bullet linear and angular factors so the game behaves like a 2D side-view game while still running in a 3D physics system. The terrain uses a Bullet heightfield shape, projectiles use spherical rigid bodies, and tanks use box-based rigid bodies. Friction and restitution were tuned so shells bounce, roll, and slow down on hills, while tanks still keep enough traction to remain controllable. Tank-versus-tank and

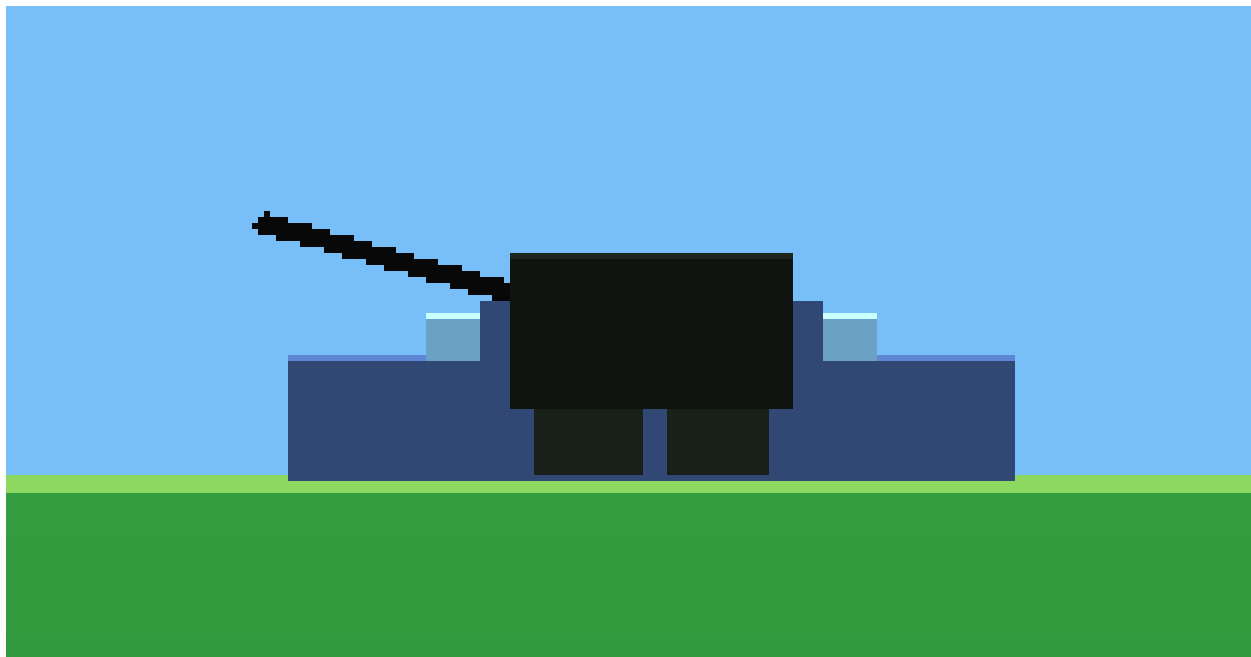


Figure 2: Tank construction from simple primitive components: lower hull, upper body, turret base, and barrel, paired with a simplified compound Bullet collision body.

projectile-versus-tank impacts also use custom response logic on top of Bullet so the contacts feel more readable and elastic in gameplay.

Exercise 4

Novel mechanics: The main novel mechanic is destructible terrain. When shells hit the ground, they carve craters into the heightfield, which changes both the visible ground and the collision surface during the match. That means each exchange of fire changes movement, cover, line of sight, and the danger of falling off the map. The two weapon types create different tactical choices: rapid fire is weaker but frequent, while heavy fire is slower, stronger, and changes the terrain more dramatically. Tanks can also jump, which adds another movement option beyond standard artillery aiming.

Exercise 5

How this could become a full game: This demo could be expanded into a full competitive artillery game. The clearest direction would be local or online multiplayer, with multiple arenas, more weapon types, rounds, scoring, menus, audio, and visual polish. It could also support single-player with AI opponents, power-ups, environmental hazards, and more advanced terrain generation. The strongest fit is a competitive multiplayer artillery game, but it could also work well as a party game because the same-keyboard format is already simple and accessible.

Exercise 6

Critique of the demo and Bullet: The demo already shows the core gameplay loop, but some interactions are still rough. Tanks can behave awkwardly on heavily deformed terrain, and mixing custom movement with physics-based collision is difficult to tune. Bullet is very good for rigid body simulation, bouncing, and terrain collision, but it does not automatically provide “game feel” behavior such as smooth vehicle traversal over dynamic terrain or highly readable arcade knockback. For a final version, I would want more direct support for stable tracked-vehicle style motion on changing terrain and better tools for combining physical realism with consistent gameplay control.

Exercise 7

Should the physics be changed for fun or playability? Yes. A final version would likely need physics to be adjusted further in the same way Rocket League bends realism for better playability. Purely realistic motion is often less fun than motion that is predictable, readable, and skill-based. Tank traction, knockback, and crater traversal would likely need to stay partially authored instead of fully realistic. I would keep Bullet for the base simulation, but I would continue overriding or tuning parts of the physics whenever realism makes the game less fair or less enjoyable.